2026-5-16 C++list--底层实现

- 1.list的构造
  - 1.1定义结点结构
  - 1.2模拟迭代器行为
    - 1.2.1第一阶段迭代器：初步构造迭代器
    - 1.2.2 第二阶段迭代器：解决const迭代器的问题
    - 1.2.3第三阶段迭代器：添加模板参数合并两个迭代器
  - 1.3将结点封装成list
- 2.默认成员函数
  - 2.1 构造函数
    - 2.1.1 多参数构造
  - 2.1析构函数
  - 2.2 拷贝构造
  - 2.3 赋值重载
- 3.修改器
  - 3.1 insert插入函数
  - 3.2 erase删除函数
  - 3.3 push插入函数
  - 3.4 pop弹出函数
  - 3.5 clear清除函数
- 总结

list的底层结构是带头结点的双向循环链表，其不支持随机访问，访问某个元素效率为O(N),在任意位置插入和删除的效率高不涉及到元素的移动，时间复杂度为O(1)。六大组件围绕着数据结构开始，遍历用迭代器来模拟。

1.list的构造

1.1定义结点结构

```
template<class T>
struct list_node {//是对list_node的封装
    T _data;
    list_node<T>* _next;
    list_node<T>* _prev;

    list_node(const T& x = T())
```

```
        : _data(x)
        , _next(nullptr)
        , _prev(nullptr)
    {
    }
};
```

1.2模拟迭代器行为

在list结构中，想要实现迭代器并没有string和vector一样那么简单，上面二者的底层都是数组，首元素地址++即可依次遍历，而list就不一样了，这个东西的底层是链表，在内存中的存储就是不连续的，遍历元素不能单靠简单的++来完成，所以对于list我们需要模拟list的行为，对其进行封装来实现出迭代器的效果来。

1.2.1第一阶段迭代器：初步构造迭代器

迭代器行为是普通指针访问数组的行为，node解引用是结构体，地址之间不连续的问题。
可以修改指向内容的数据。

```
template<class T>
struct __list_iterator {
    typedef list_node<T> Node;
    Node* _node;

    __list_iterator(Node* node) : _node(node) {}

    T& operator*() { return _node->_data; }

    // 前置++
    __list_iterator<T>& operator++() {
        _node = _node->_next;
        return *this;
    }
    // 后置++
    __list_iterator<T> operator++(int) {
        __list_iterator<T> tmp(*this);
        _node = _node->_next;
        return tmp;
    }
    // 前置--
    __list_iterator<T>& operator--() {
        _node = _node->_prev;
```

```
        return *this;
    }
    // 后置--
    __list_iterator<T> operator--(int) {
        __list_iterator<T> tmp(*this);
        _node = _node->_prev;
        return tmp;
    }

    bool operator!=(const __list_iterator<T>& it) const {
        return _node != it._node;
    }
    bool operator==(const __list_iterator<T>& it) const {
        return _node == it._node;
    }
};
```

迭代器有一个指针，迭代器要不要写析构。这个结点不属于迭代器，这个结点属于链表，迭代器只有使用权，实际上是是list管理的，不是有结点就要析构。
正常不需要写析构就意味着不需要写拷贝构造和赋值重载。这里的资源不属于它，默认生成浅拷贝就可。不能说有指针指向资源就要析构。
vector的迭代器也不一定是原生指针。

1.2.2 第二阶段迭代器：解决const迭代器的问题

迭代器修改是通过`operate*`来修改,重新定义const迭代器。
指向内容的数据不可修改。这里本质上就只是修改了解引用的重载。

```
template<class T>
struct __list_const_iterator {
    typedef list_node<T> Node;
    Node* _node;

    __list_const_iterator(Node* node) : _node(node) {}

    //控制operate*
    const T& operator*() { return _node->_data; }
    // 前置++
    __list_const_iterator<T>& operator++() {
        _node = _node->_next;
        return *this;
    }
    // 后置++
```

```
    __list_const_iterator<T> operator++(int) {
        __list_const_iterator<T> tmp(*this);
        _node = _node->_next;
        return tmp;
    }
    // 前置--
    __list_const_iterator<T>& operator--() {
        _node = _node->_prev;
        return *this;
    }
    // 后置--
    __list_const_iterator<T> operator--(int) {
        __list_const_iterator<T> tmp(*this);
        _node = _node->_prev;
        return tmp;
    }

    bool operator!=(const __list_const_iterator<T>& it) const {
        return _node != it._node;
    }
    bool operator==(const __list_const_iterator<T>& it) const {
        return _node == it._node;
    }
};
```

这里不能和vector一样重载，
不能实现，因为const迭代器本身，导致迭代器不能++

```
//不能实现，因为const迭代器本身，导致迭代器不能++
//typedef const __list_iterator<T> iterator;
```

也不能重载：
下述重载会导致调用的迭代器本身无法修改，因为不得不在前面加上`const`，该对象不能调用++，只能重新搞一个类型。

```
const T& operator*() const
{ return _node->_data; }
```

1.2.3第三阶段迭代器：添加模板参数合并两个迭代器

但是有点太啰嗦了，那这里我们就可以进行修改,将两个重复的定义合并到一起。

```
template<class T,class Ref,class Ptr>//这里还要一个参数
struct __list_iterator {
    typedef list_node<T> Node;
    //为了避免将下面的所有模板参数都进行修改，我们直接做定义
    typedef __list_iterator<T, Ref,Ptr> Self;
    Node* _node;
    __list_iterator(Node* node) : _node(node) {}

    //控制operate*
    Ref operator*() { return _node->_data; }//本质上是控制返回参数不一样
    //增加一个模板参数
    // 前置++
    Ptr operator->() {
        return &_node->_data;//这里就是把data的地址取出来
        //就是it->->_col实际要这样写
    }
    Self& operator++() {
        _node = _node->_next;
        return *this;
    }
    // 后置++
    Self operator++(int) {
        Self tmp(*this);
        _node = _node->_next;
        return tmp;
    }
    // 前置--
    Self& operator--() {
        _node = _node->_prev;
        return *this;
    }
    // 后置--
    Self operator--(int) {
        Self tmp(*this);
        _node = _node->_prev;
        return tmp;
    }

    bool operator!=(const Self& it) const {
        return _node != it._node;
    }
    bool operator==(const Self& it) const {
        return _node == it._node;
    }
};
```

1.3将结点封装成list

```
template<class T>
class list {
typedef list_node<T> Node;
public:
    typedef __list_iterator<T,T&,T*> iterator;
    typedef __list_iterator<T,const T&,const T*> const_iterator;
    //同一个类模板实例化的两个模板参数
    //不能实现，因为const迭代器本身，导致迭代器不能++
    //typedef const __list_iterator<T> iterator;
    //typedef __list_const_iterator<T> const_iterator;//二者是两个不同的对象
    //普通迭代器
    iterator begin() {
        return iterator(_head->_next);
    }
    iterator end() {
        return iterator(_head);
    }
    //const迭代器
    const_iterator begin() const {
        return const_iterator(_head->_next);
    }

    const_iterator end() const {
        return const_iterator(_head);//返回const迭代器
    }
   void empty_init() {
        _head = new Node;
        _head->_next = _head;
        _head->_prev = _head;
    }
    list() {
        empty_init();
    }
    private:
        Node* _head;
        size_t _size = 0;//否则遍历也行
    };
}
```

测试用例：

```
namespace ZL {
    void text_list01() {
        ZL::list<int> lt;
        lt.push_back(1);
        lt.push_back(2);
        lt.push_back(3);
        lt.push_back(4);

        list<int>::iterator it = lt.begin();
        while (it != lt.end()) {
            cout << *it << " ";
            ++it;
        }
        cout << endl;
    }
}
```

2.默认成员函数

2.1 构造函数

普通构造上面已经说过了，接下来我们说：

2.1.1 多参数构造

```
list(std::initializer_list<T> il) {
    empty_init();
    for (const auto& e : il) {
        push_back(e);
    }
}
```

2.1析构函数

```
//析构
~list() {
    clear();
    delete _head;
    _head = nullptr;
}
```

2.2 拷贝构造

```
//拷贝构造
list(const list<T>& lt) {
    empty_init();
    for (const auto& e : lt) {
        push_back(e);
    }
}
```

2.3 赋值重载

赋值重载的传统写法：

```
list<T>& operator=(const list<T>& lt) {
  if (this != &lt) {
    clear();
    for (const auto& e : lt) {
        push_back(e);
    }
}
    return *this;
}
```

赋值重载的现代写法：

```
 void swap(list<T>& lt) {
     std::swap(_head, lt._head);
     std::swap(_size, lt._size);
 }
 //lt1(lt2);
list<T>& operator=(list<T> lt) {
    swap(lt);//将this指针（lt2)的值交换给lt1，来完成构造
    return *this;
}
```

3.修改器

3.1 insert插入函数

这里就是和双向链表的插入删除相似了。

```
iterator insert(iterator pos,const T& val) {//用迭代器屏蔽底层的细节
    Node* cur = pos._node;//体现出iterator封装的便利性
    Node* newNode = new Node(val);
    Node* prev = cur->_prev;
    prev->_next = newNode;
    newNode->_prev = prev;
    newNode->_next = cur;
    cur->_prev = newNode;
    _size++;
    return (newNode);
}
```

3.2 erase删除函数

```
iterator erase(iterator pos) {
    Node* cur = pos._node;
    Node* prev = cur->_prev;
    Node* next = cur->_next;
    prev->_next = next;
    next->_prev = prev;
    delete cur;
    //return next;构造隐式类型转化
    _size--;
    return (next);//构造匿名对象
}
```

3.3 push插入函数

正常应该这么写：

```
 void push_back(const T& x) {
     Node* newnode = new Node(x);
     Node* tail = _head->_prev;
     tail->_next = newnode;
     newnode->_prev = tail;
     newnode->_next = _head;
     _head->_prev = newnode;
 }
```

但是实际上我们可以对insert进行复用：

```
void push_back(const T& x) {
    insert(end(), x);
}
void push_front(const T& x) {
    insert(begin(), x);
}
```

3.4 pop弹出函数

```
void pop_back( ) {
    assert(!empty());
    erase(--end());
}
void pop_front( ) {
    assert(!empty());
    erase(begin());
}
```

3.5 clear清除函数

```
void clear() {
    iterator it = begin();//用迭代器拷贝it,浅拷贝
    while (it != end()) {
        it = erase(it);
    }
    _size=0;
}
```

总结

分类	核心内容	关键点/代码
底层结构	带头结点的双向循环链表	插入删除 O(1)，不支持随机访问
节点定义	`list_node<T>`	`_data`，`_next`，`_prev`，构造函数支持默认值
迭代器设计	三个阶段演进	普通迭代器 → const迭代器 → 模板参数合并

分类	核心内容	关键点/代码
迭代器核心操作	`operator*`, `operator->`, `++`, `--`, `!=`, `==`	通过 `Ref` 和 `Ptr` 模板参数区分 const 版本
list 类成员	`_head` (头结点), `_size` (元素个数)	哨兵位头结点，双向循环
构造/析构	`list()`, `~list()`, `empty_init()`	头结点自循环，`clear()` + 释放头结点
拷贝构造	`list(const list& lt)`	复用 `empty_init()` + `push_back`
赋值重载	传统写法 + 现代写法 (copy-and-swap)	现代写法：传值参数 + `swap`
插入操作	`insert`, `push_back`, `push_front`	`insert` 返回新节点迭代器，其他复用 `insert`
删除操作	`erase`, `pop_back`, `pop_front`	`erase` 返回下一节点迭代器，防止迭代器失效
清空操作	`clear()`	遍历 `erase`，重置 `_size = 0`
多参数构造	`initializer_list<T>`	支持 `list<int> lt = {1,2,3}`
迭代器接口	`begin()`, `end()`，及 const 版本	普通返回 `iterator`，const 返回 `const_iterator`